Experiment 3: Pipelined AES-256 Encryption Engine

FPGA Build Challenge 2026 - Intermediate level - AMD PYNQ-Z2 (Zynq-7000 XC7Z020)

1. Objective

AES (Advanced Encryption Standard) is the block cipher used to protect most data today, from Wi-Fi and HTTPS to disk encryption. It works on 128-bit blocks; AES-256 uses a 256-bit key and 14 rounds. Doing AES in software takes the processor many instructions per block. The objective of this experiment is to build AES-256 encryption and decryption as hardware in the FPGA, with one pipeline stage per round so that a new block can enter every clock cycle, and to use it from the ARM processor over AXI4-Lite. The design is checked against the official FIPS-197 test vector and against a software AES library.

2. Block Diagram

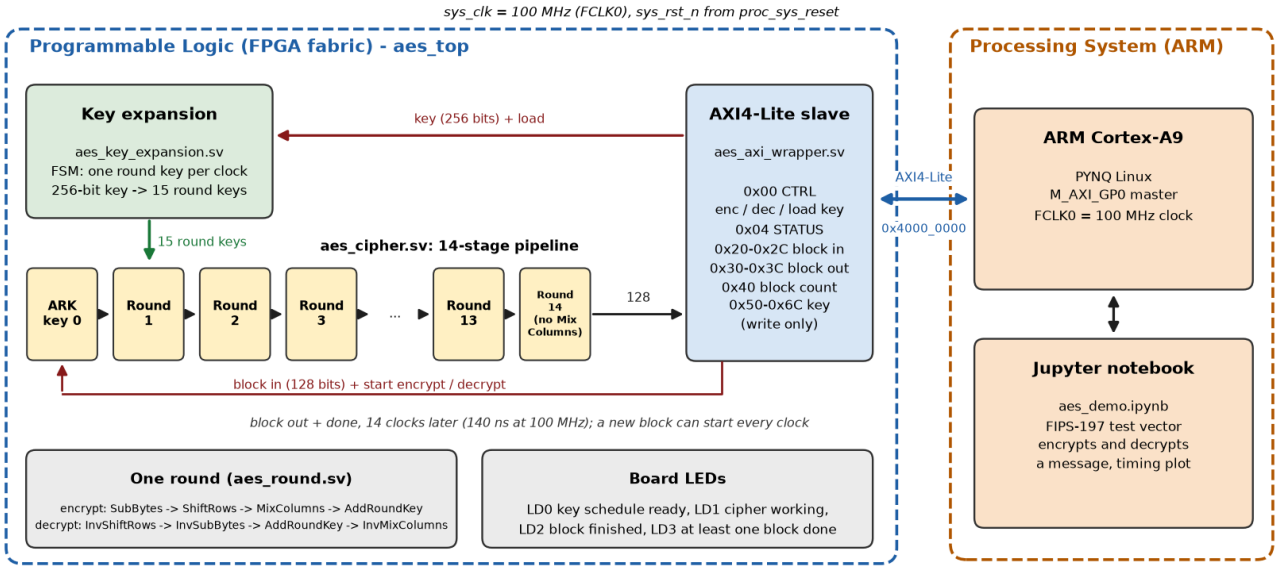


Figure 1 - System block diagram

The processor writes the 256-bit key into write-only registers and starts the key expansion, which computes the 15 round keys (one per clock) and keeps them in registers. It then writes a 128-bit block and starts an encryption or a decryption. The block is XORed with the first round key and passes through 14 round stages; each stage is one clock, so the result appears 14 clocks (140 ns) later. The AXI wrapper catches the result when the last stage signals valid, and the processor reads it back.

3. RTL Design

3.1 Module hierarchy

```
system_wrapper                (Vivado block design)
 |-- processing_system7_0     ARM Cortex-A9, gives FCLK0 = 100 MHz and M_AXI_GP0
 |-- rst_ps7_0_100M           reset synchroniser
 |-- ps7_0_axi_periph         AXI interconnect / protocol converter
 +-- aes_top_0                aes_top.sv  (block counter, LEDs)
      |-- u_axi_wrapper       aes_axi_wrapper.sv
      |-- u_key_expansion     aes_key_expansion.sv  (4 x aes_sbox)
      +-- u_cipher            aes_cipher.sv
           |-- gen_round[1..13].u_round   aes_round.sv (with MixColumns)
           +-- u_final_round              aes_round.sv (no MixColumns)
                |-- aes_sub_bytes  -> 16 x aes_sbox
                |-- aes_shift_rows
                |-- aes_mix_columns
                +-- (AddRoundKey = XOR with the round key)
```

3.2 Modules

Module (file)	What it does
aes_sbox (aes_sbox.sv)	The AES S-box and inverse S-box as 256-entry lookup tables: one byte in, one byte out.
aes_sub_bytes (aes_sub_bytes.sv)	Applies the S-box (or inverse S-box) to all 16 bytes of the state.
aes_shift_rows (aes_shift_rows.sv)	Rotates row r of the 4x4 byte state left by r bytes (right for decryption). Only wiring, no logic.
aes_mix_columns (aes_mix_columns.sv)	Multiplies each column by a fixed matrix in GF(2^8) using "xtime" (shift and conditional XOR with 0x1B); the inverse matrix is used for decryption.
aes_add_round_key	XOR of the state with the 128-bit round key.
aes_round (aes_round.sv)	One round, with both paths built in parallel: encrypt = SubBytes, ShiftRows, MixColumns, AddRoundKey; decrypt = InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns. A multiplexer picks one and a 128-bit register stores it: this register is the pipeline stage. The final round has no MixColumns.
aes_cipher (aes_cipher.sv)	Initial AddRoundKey, 13 normal rounds and the final round chained as a 14-stage pipeline. The encrypt/decrypt flag travels down the pipeline with the data, and decryption uses the round keys in reverse order.
aes_key_expansion (aes_key_expansion.sv)	State machine (IDLE, EXPAND, READY). Round keys 0 and 1 are the two halves of the key; keys 2 to 14 are computed one per clock with RotWord, SubWord (4 S-boxes) and the Rcon constant, as in FIPS-197.
aes_axi_wrapper (aes_axi_wrapper.sv)	AXI4-Lite slave: control pulses, status, the 4 input words, 4 result words (captured when the cipher signals done), the block counter and 8 key words that read back as zero.
aes_top (aes_top.sv)	Connects the blocks, only starts the cipher when a key schedule is ready, counts finished blocks and stretches the busy/done pulses to about 84 ms so they are visible on LD1 and LD2.

3.3 Register map (base address 0x4000_0000)

Offset	Name	Access	Meaning
0x00	CTRL	W	bit 0 start encrypt, bit 1 start decrypt, bit 2 load the key
0x04	STATUS	R	bit 0 busy, bit 1 done, bit 2 key schedule ready
0x20-0x2C	BLOCK IN	R/W	128-bit input block, most significant word first
0x30-0x3C	BLOCK OUT	R	128-bit result
0x40	BLOCKS	R	number of blocks processed
0x50-0x6C	KEY	W	256-bit key, most significant word first; reads return 0

3.4 RTL schematics

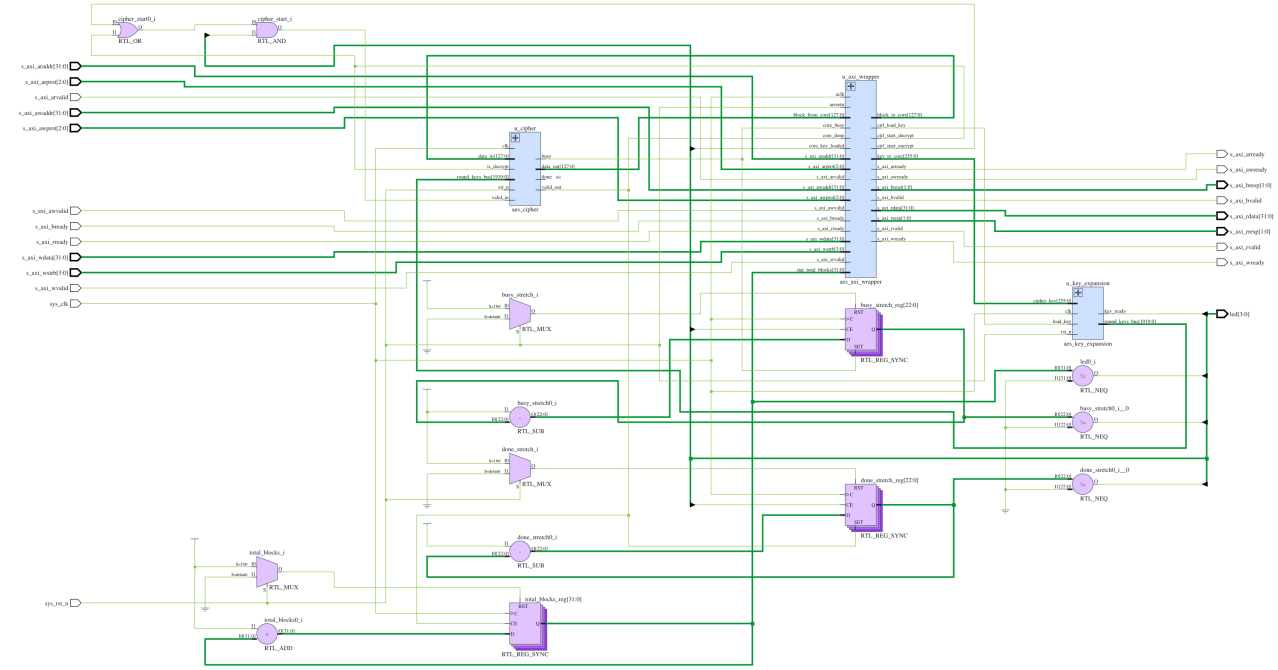


Figure 2 - Elaborated RTL schematic of aes_top (Vivado)

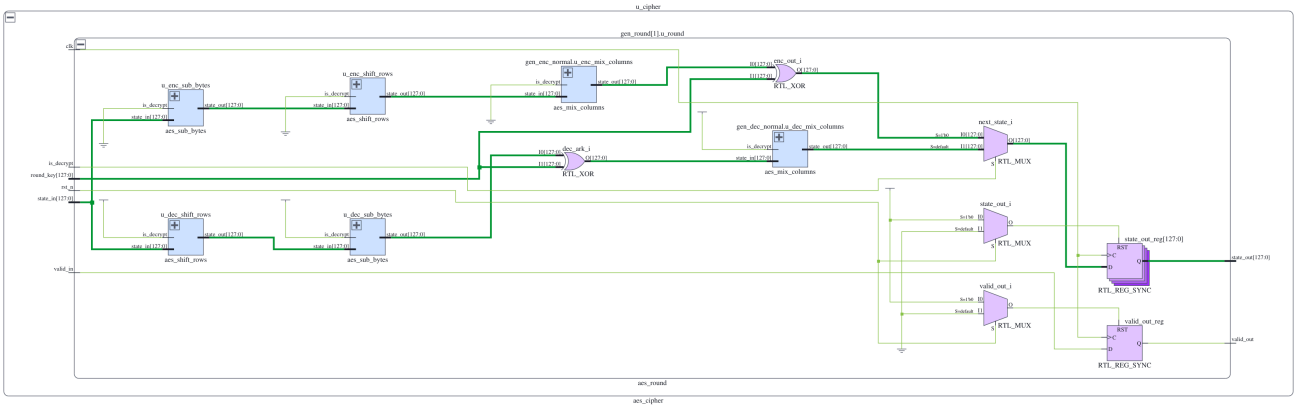


Figure 3 - One round (gen_round[1]): encrypt path (top) and decrypt path (bottom) feed the pipeline register

4. Simulation Results

The design was simulated with Icarus Verilog using the testbench *aes_tb.sv* with the FIPS-197 appendix C.3 AES-256 example. All checks passed:

```
key schedule ready (status = 00000004)
plaintext  00112233445566778899aabbccddeeff
ciphertext 8ea2b7ca516745bfeafc49904b496089
[PASS] encryption matches the FIPS-197 test vector
decrypted  00112233445566778899aabbccddeeff
[PASS] decryption recovered the original block
[PASS] the key registers read back as zero
blocks processed: 2
```

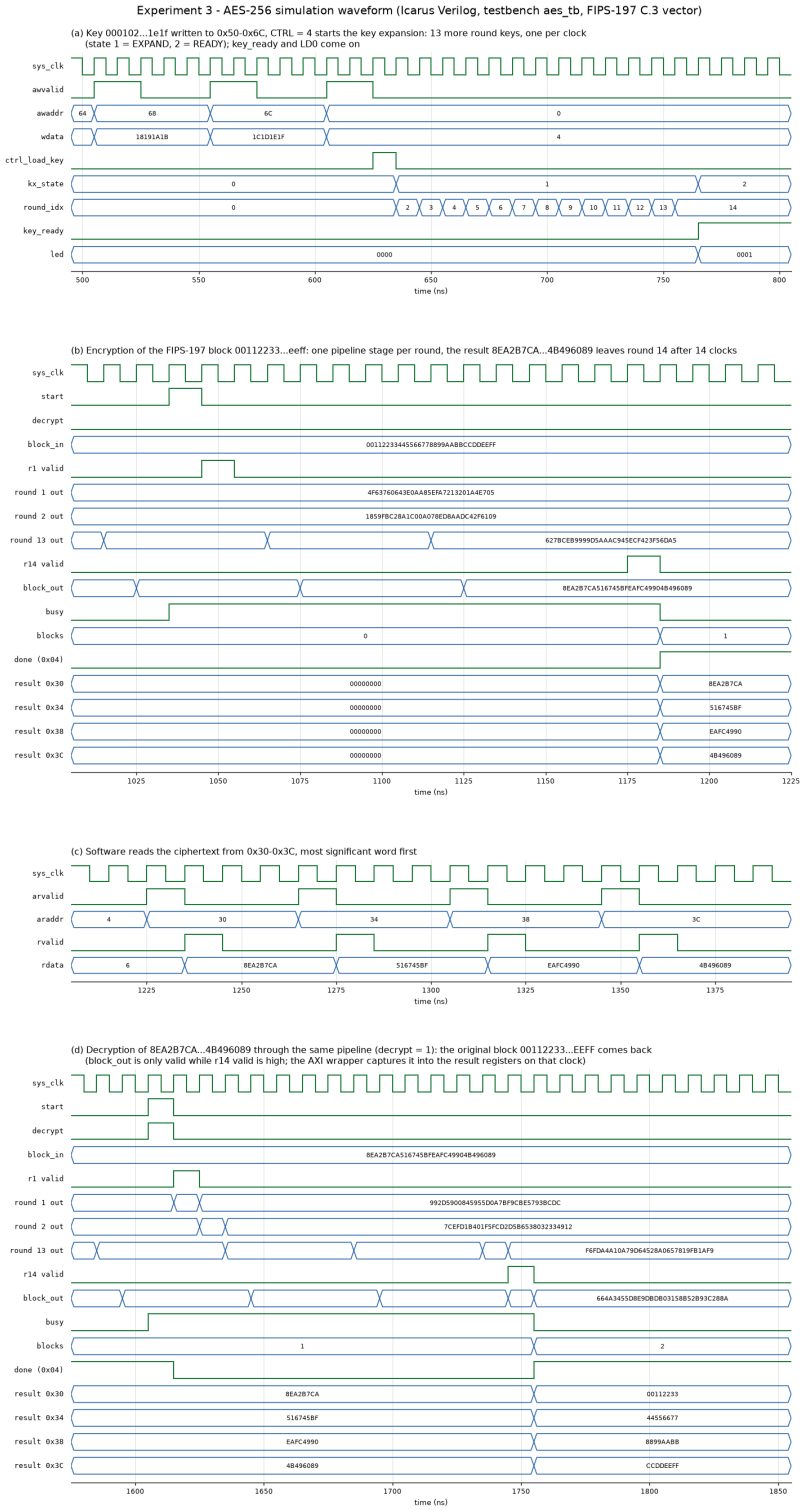


Figure 4 - Simulation waveform (details in Simulation/simulation_report.pdf)

5. Hardware Implementation

5.1 Build

The design was built with Vivado 2026.1 for the xc7z020clg400-1 device using build.tcl. The constraints file (aes.xdc) assigns the four LEDs. The 14 unrolled rounds each contain 16 S-boxes for encryption and 16 inverse S-boxes for decryption plus the 4 S-boxes of the key expansion - about 450 lookup tables of 256 bytes - so the design uses about half of the FPGA's LUTs, most of them (with the F7/F8 multiplexers) for the S-boxes.

Resource	Used	Available	Utilisation
Slice LUTs	26,763	53,200	50.31 %
Slice registers (flip-flops)	4,922	106,400	4.63 %
F7 / F8 multiplexers	5,233 / 804	26,600 / 13,300	19.67 % / 6.05 %
Block RAM tiles	0	140	0 %
Bonded I/O pins	4	125	3.20 %
Global clock buffers	1	32	3.13 %

Timing (100 MHz clock)	Value	Status
Worst setup slack (WNS)	+0.273 ns	**met**
Worst hold slack (WHS)	+0.010 ns	**met**
Failing endpoints	0 of 12,533	**met**

Each pipeline stage does a full round (S-box, MixColumns and XOR) in one 10 ns clock, which is why the setup slack is small but positive.

5.2 Test on the board

The bitstream was loaded into the PYNQ-Z2 and the engine was driven through its AXI registers exactly as the notebook aes_demo.ipynb does it. The results were compared with the Python "cryptography" library (AES-256, ECB mode). 120 blocks were processed in total.

Check	Detail	Result
FIPS-197 C.3 encryption	got 8ea2b7ca516745bfeafc49904b496089	**PASS**
FIPS-197 C.3 decryption	got 00112233445566778899aabbccddeeff	**PASS**
Key registers read back as zero	00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000	**PASS**
Message ciphertext equals software AES	3 blocks	**PASS**
Message decrypts back to the text	'FPGA hardware encryption on the PYNQ-Z2 board!!!'	**PASS**
Random blocks encrypted correctly	40 of 40	**PASS**
Random blocks decrypted correctly	40 of 40	**PASS**
Avalanche blocks equal software AES	32 blocks	**PASS**
Block counter	120 blocks counted, 120 sent	**PASS**

Avalanche effect: flipping one plaintext bit changed on average 65.7 of 128 ciphertext bits (min 52, max 78, 32 flips). A good cipher changes about half of the output bits when one input bit changes, so that similar messages give unrelated ciphertexts.

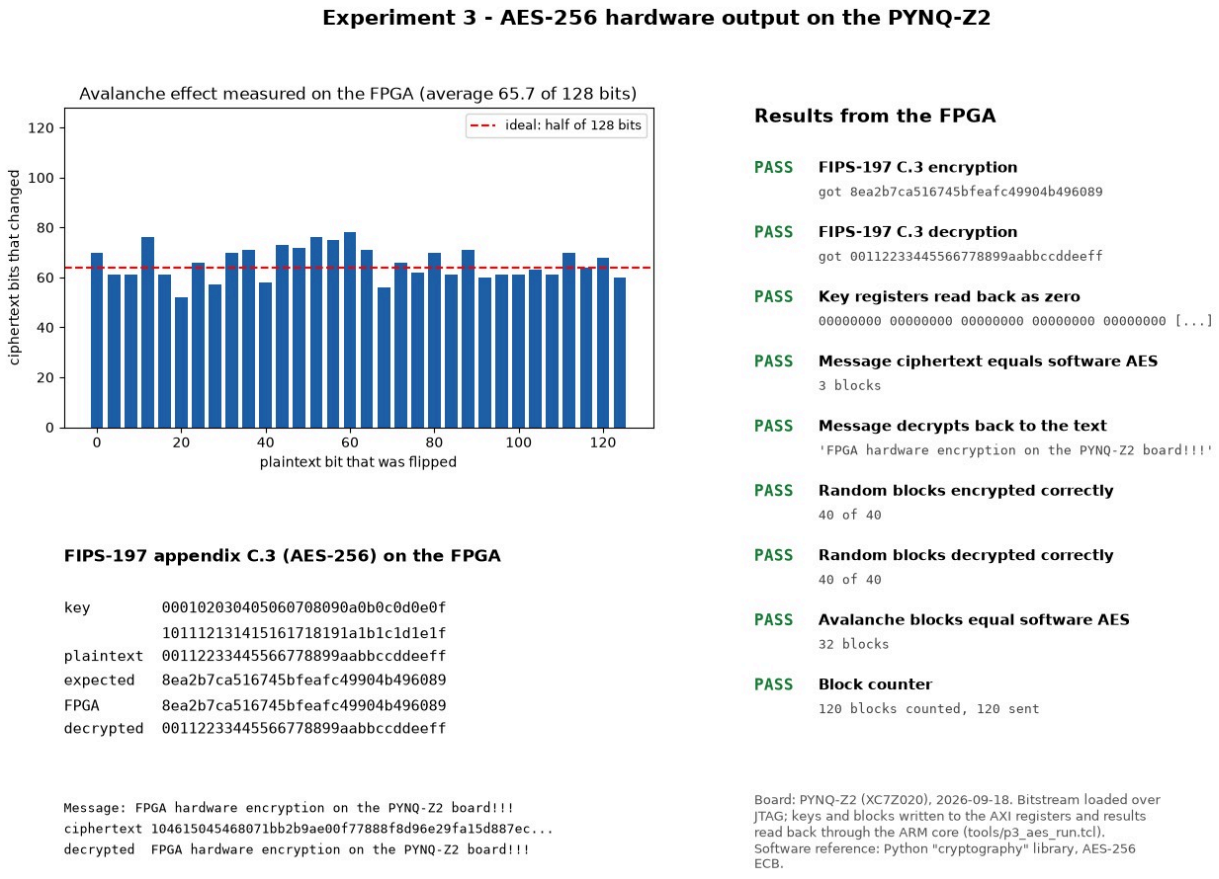


Figure 5 - AES-256 results measured on the FPGA

Run of the Jupyter notebook on the board

The steps of the notebook aes_demo.ipynb (Jupyter folder) were also run on the PYNQ-Z2 with PYNQ, using the same bitstream. tools/execute_all_on_pynq.py runs them on the board over SSH and copies the log and the graph back, and tools/embed_nb_outputs.py puts them into the notebook. It repeated the FIPS-197 test (ciphertext 8ea2b7ca516745bfeafc49904b496089, decryption returns the plaintext) and encrypted and decrypted the 48-byte message (3 blocks), which came back unchanged. Through PYNQ one block took 0.27 ms including the Python register accesses; the cipher itself needs only 140 ns. The log and the graph the notebook drew are in Jupyter/output.

```
[*] Flashing FPGA bitstream for Experiment 3 (AES)...
[+] FPGA Flashed! DONE LED active.
Overlay loaded, AES engine at 0x40000000
plaintext   00112233445566778899aabbccddeeff
ciphertext  8ea2b7ca516745bfeafc49904b496089
expected    8ea2b7ca516745bfeafc49904b496089
decrypted   00112233445566778899aabbccddeeff
encryption matches FIPS-197: PASS
decryption returns the input: PASS
message   : FPGA hardware encryption on the PYNQ-Z2 board!!!
encrypted : 104615045468071bb2b9ae00f77888f8d96e29fa15d887ecfe101d854140b432be385b36418697c76e21e0d940642217
decrypted : FPGA hardware encryption on the PYNQ-Z2 board!!!
round trip: PASS
3 blocks encrypted in 0.8 ms (0.27 ms per block)
blocks processed since power up: 8
```

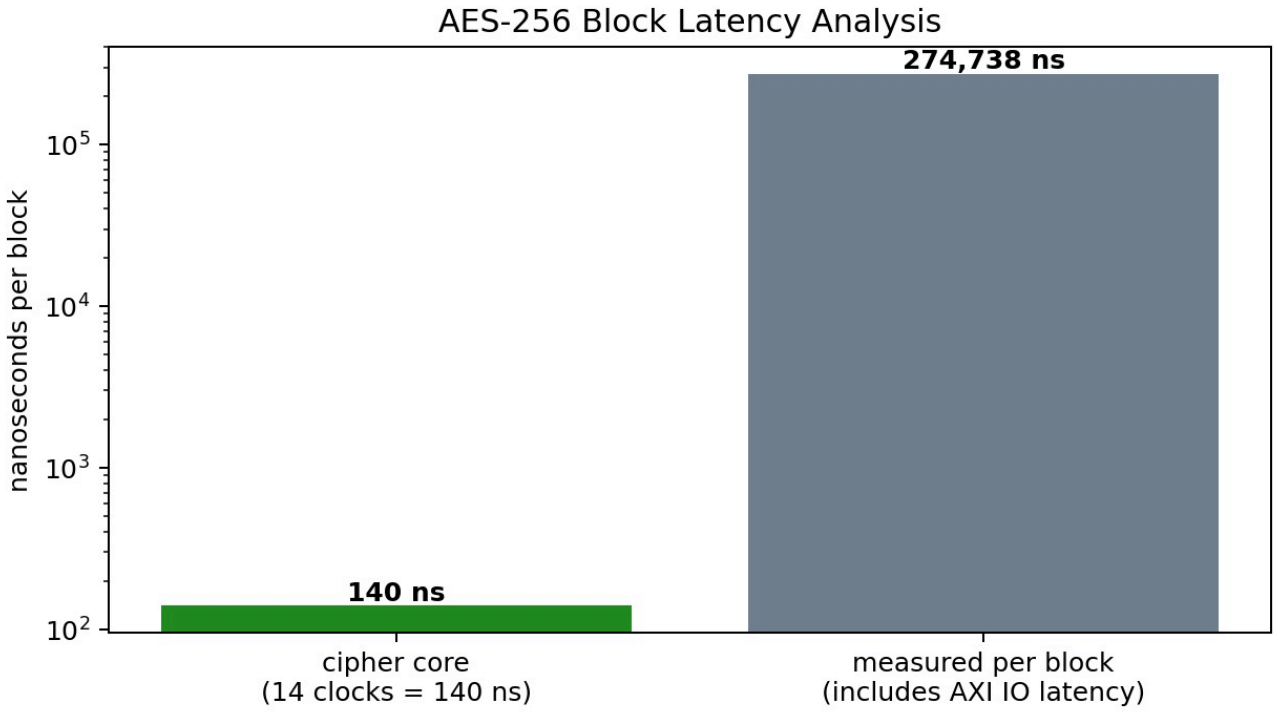


Figure 6 - Graph drawn by aes_demo.ipynb on the board

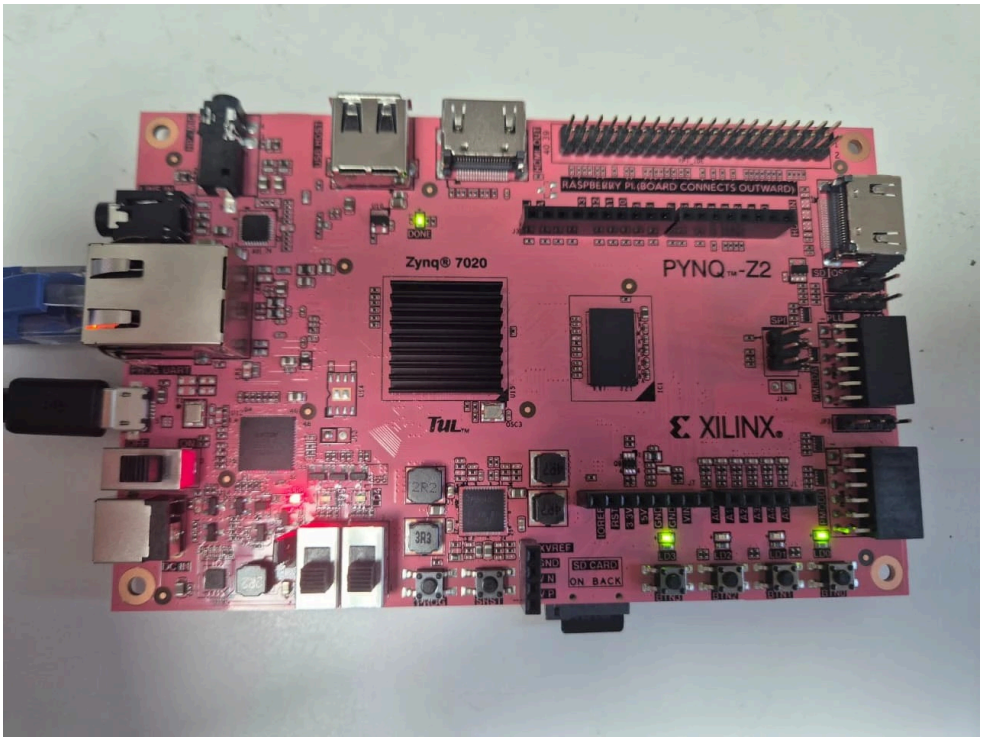


Figure 7 - PYNQ-Z2 board setup: micro-USB (power and JTAG/UART), Ethernet to the PC, PYNQ Linux on the SD card

The cipher itself needs 14 clocks (140 ns) per block, which is about 114 MB/s for one block at a time and up to 1.6 GB/s if a new block were started every clock. Through AXI4-Lite the processor has to write four words, start the engine, poll the status and read four words for every block, so in practice the speed is set by these register accesses, not by the cipher.

6. Applications

- **Secure communication:** encrypting network packets (VPN, TLS, Wi-Fi WPA2/WPA3 use AES) at line rate without loading the processor.
- **Storage encryption:** encrypting disks, SD cards and flash in embedded systems.
- **IoT and sensor networks:** protecting sensor readings before they are sent over radio.
- **Secure boot and firmware protection:** decrypting firmware images inside the device.

7. Conclusion

A pipelined AES-256 engine was designed, simulated and implemented on the PYNQ-Z2. It meets timing at 100 MHz, produces the FIPS-197 reference result, matched a software AES library on all 120 test blocks on the board, and keeps the key write-only.

Learning outcomes: the structure of AES (SubBytes, ShiftRows, MixColumns, AddRoundKey and the key schedule); arithmetic in GF(2^8); how pipelining trades area for throughput (one round per stage, a new block every clock); reusing one datapath for encryption and decryption; and verifying hardware against a standard test vector and a software reference.

Future scope: add AXI-Stream and DMA so that blocks can really enter every clock; add block cipher modes such as CTR or GCM in hardware (ECB alone is not secure for long messages); save area by sharing S-boxes or by building only the encryption path; and add protection against side-channel attacks.